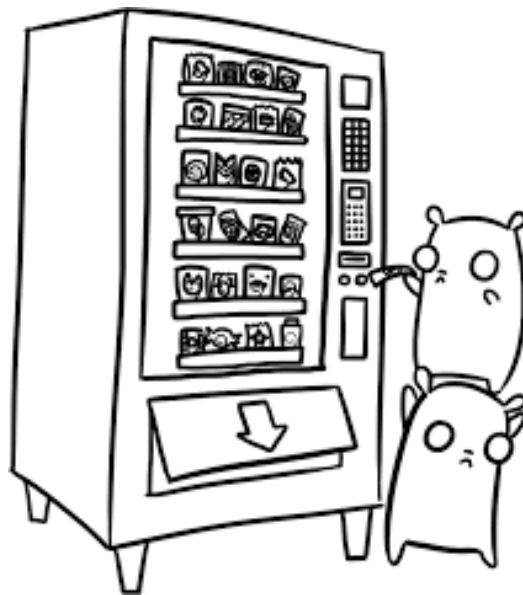


MÁQUINA DE VENDING



ÍNDICE

Descripción del proyecto	3
Diseño	3
Funcionalidad requerida	6
Pruebas	7
Consejos y consideraciones	12
Informe	14
Evaluación del proyecto	16
Entrega	17
Anexo: Defensa del proyecto	18

1. DESCRIPCIÓN DEL PROYECTO

Realizar un programa que simule el **funcionamiento de una máquina expendedora (máquina de vending)**, permitiendo realizar acciones como comprar un producto si hay crédito y stock suficiente, devolver cambio, añadir nuevos productos, eliminar productos del stock...

La máquina tendrá dos modos de funcionamiento; uno accesible para cualquier usuario y otro accesible únicamente para los administradores, que se encontrará protegido por contraseña.

El objetivo de este proyecto no es únicamente lograr un programa que funcione, sino que se espera que el producto final tenga un diseño sólido y bien estructurado. Para ello se espera que a lo largo del desarrollo se tomen decisiones sobre la organización del código en clases y paquetes, qué estructuras de datos son más óptimas para cada tarea o cómo conectar los distintos componentes del sistema.

2. DISEÑO

El sistema estará dividido en, al menos, dos paquetes con responsabilidades claramente diferenciadas:

- Un paquete para los productos, que contendrá la jerarquía de clases que modela lo que vende la máquina.
- Un paquete para la lógica, que contendrá todas las clases necesarias para el funcionamiento del sistema (la máquina).

Jerarquía de productos

La máquina permite vender dos tipos de productos: bebidas y snacks. De todos los productos se conoce su id y su nombre y se sabe que tendrán un precio, que se deberá calcular cuando se conozcan los detalles necesarios.

Además, de las bebidas se conoce su tamaño en mililitros y si se trata de una bebida azucarada o no. Mientras que de los snacks se conoce su tamaño (pudiendo ser este "L", "M" o "S", en referencia a "Large", "Medium" y "Small").

Las reglas para conocer el precio de venta de cada producto son las siguientes:

Bebidas (precio base: 1,00 €)

- Azucarada y ≥ 500 ml $\rightarrow$ recargo del 20 % sobre el precio base.
- Azucarada y < 500 ml $\rightarrow$ recargo del 10 % sobre el precio base.
- No azucarada y < 500 ml $\rightarrow$ descuento del 15 % sobre el precio base.
- Sin azúcar y ≥ 500 ml $\rightarrow$ precio base.

Snacks (precio base: 1,50 €)

- Tamaño L $\rightarrow$ recargo del 20 % sobre el precio base.
- Tamaño M $\rightarrow$ precio base.
- Tamaño S $\rightarrow$ descuento del 10 % sobre el precio base.

Decisión de diseño: el precio

Cada tipo de producto calcula su precio de forma distinta según sus propios atributos, esto afectará a donde debe realizarse este cálculo dentro de la jerarquía de clases, que deberá atender a lo que tienen en común y lo que diferencia a los tipos de producto.

Advertencia: Precisión decimal

Los precios son dinero, por lo que no es conveniente utilizar los tipos *double* y *float* para los cálculos. Java cuenta con la clase ***BigDecimal*** para trabajar específicamente con aritmética decimal exacta. Investiga sobre el funcionamiento de esta y utilízala.

Gestión del stock

La máquina tiene una cuadrícula de ranuras identificadas por filas y columnas. Las máquinas standard del fabricante en cuestión cuentan con 4 filas (A-D) y 4 columnas (1-4). Cada ranura puede estar vacía o contener una cantidad determinada de un producto. Siempre se debe tener en cuenta que el máximo de unidades por ranura es de 10.

Decisión de diseño: el stock

Valora cuál es la estructura más adecuada para el stock. Ten en cuenta que es necesario asociar cada código de ranura ("A1", "B3" ...) con el producto que hay en ella y su cantidad.

Decisión de diseño: producto y cantidad

Ten en cuenta que la cantidad no es una característica de un producto, por lo que debes decidir cómo almacenar estos datos de forma conjunta sin romper la abstracción de producto.

El sistema de stock debe ser capaz de:

- Validar que un código de ranura tiene el formato correcto antes de operar con él.
- Añadir un producto en una ranura libre junto a la cantidad disponible.
Nota: No se podrá añadir un producto que ya se encuentre en otra ranura. Dos productos son iguales si tienen el mismo id.
- Modificar el stock de un producto tras una venta.
- Eliminar un producto de una ranura, haciendo que esta pase a estar vacía.
- Mostrar el listado completo de ranuras con su contenido, incluyendo aquellas que se encuentren vacías.

Gestión de monedas

La máquina mantiene un depósito físico de monedas que usa para almacenar el dinero introducido por los usuarios y dar cambio en caso de que sea necesario. Las monedas soportadas son: 2,00 €, 1,00 €, 0,50 €, 0,20 €, 0,10 € y 0,05 €.

El algoritmo para dar el cambio debe seguir una estrategia **greedy**: intentar usar siempre la moneda de mayor valor posible, en orden descendente, hasta completar el importe exacto. Si al final no es posible dar el cambio exacto, la venta debe cancelarse (el sistema se quedará con el crédito e indicará que debe introducirse el importe exacto o solicitar la devolución del crédito).

Decisión de diseño: el deposito

Valora cual es la estructura más adecuada para el depósito. Ten en cuenta que es necesario saber cuántas monedas hay de cada denominación y, para ello, será necesario relacionar denominación y cantidad.

Decisión de diseño: estrategia greedy

La estrategia **greedy** ya ha sido implementada en clase, una vez que sepas lo que es revisa los ejercicios realizados.

Existen varias formas de llevar a cabo el proceso del cambio, pero en todas ellas es necesario recorrer las denominaciones en un orden concreto. Analiza las ventajas de cada una e implementa la que consideres se ajusta mejor a tu diseño.

El sistema de stock debe ser capaz de:

- Crear un deposito con una cantidad de monedas de cada tipo (la misma para todas las monedas soportadas).
- Calcular la vuelta que hay que dar tras la venta de un producto en función del crédito introducido y el coste de este.
- Mantener el deposito en un estado consistente (al dar la vuelta se quitarán monedas, mientras que las monedas introducidas por el usuario cuando se culmine una venta con éxito se deberán añadirán).
- Llevar la cuenta de las ganancias totales acumuladas con la venta de productos.
- Mostrar el estado del depósito (cantidad de monedas disponibles de cada tipo).

3. FUNCIONALIDAD REQUERIDA

Una vez desarrollados los componentes del sistema (jerarquía de productos, gestión de stock y depósito de monedas) es el momento de profundizar en las acciones que permitirá realizar la máquina.

Toda la interacción con el sistema se llevará a cabo por teclado y será guiada por pantalla a través de menús y mensajes informativos.

Modo usuario

Pantalla principal que se muestra al iniciar la máquina. Muestra el listado de productos disponibles, el crédito acumulado en cada momento y ofrece las siguientes opciones:

- **Insertar monedas:** El usuario pulsa teclas que representan monedas ($A=2\text{€}$, $B=1\text{€}$, $C=0,50\text{€}$, $D=0,20\text{€}$, $E=0,10\text{€}$, $F=0,05\text{€}$). El crédito se acumula hasta que pulsa "S".
- **Comprar producto:** El usuario introduce el código de ranura (ej: A1). El sistema valida que existe, que hay stock y que el crédito es suficiente. Si todo es correcto, procesa la venta y muestra el desglose del cambio devuelto.
- **Devolver crédito:** Devuelve al usuario el dinero acumulado sin realizar ninguna compra.
- **Modo administrador:** Solicita la contraseña; si es correcta, accede al menú de administración. (Dispone de 3 intentos, si no el sistema regresará al menú principal).

Modo administrador

Protegido por un pin de 4 dígitos alfanuméricos (PIN inicial: "1234"), se accede a él desde el modo usuario. Una vez dentro del panel de administración se presentan las siguientes opciones:

- **Reponer stock:** Añadir unidades de un producto ya disponible en la máquina. (No puede superar el máximo de unidades permitidas por ranura).
- **Nuevo producto:** Configurar una ranura vacía para que albergue un nuevo producto (se debe registrar adecuadamente la información del nuevo producto: tipo, atributos y cantidad inicial).
Nota: No se podrá añadir un producto que ya se encuentre en otra ranura. Dos productos son iguales si tienen el mismo id.
- **Eliminar producto:** Vaciar una ranura, eliminando el producto que contenía hasta el momento.
- **Ver estado de la caja:** Muestra cuantas monedas de cada denominación hay en el depósito en un momento determinado.
- **Ver recaudación total:** Muestra el total de dinero ingresado por ventas desde que se inició el sistema.
- **Ver stock detallado:** Muestra el listado completo de productos, junto a su precio y la cantidad disponible en un momento determinado.
- **Cambiar contraseña:** Permite modificar el PIN de acceso al modo administrador (el nuevo pin debe cumplir las restricciones establecidas)

4. PRUEBAS

Para facilitar las pruebas se debe implementar un método que, al ser llamado, cargue automáticamente los siguientes productos en el sistema:

Ranura — Producto	Atributos — Unidades disponibles
A1 — Cola	Bebida (P01), 330 ml, azucarada — 5 unidades
A2 — Agua	Bebida (P02), 500 ml, sin azúcar — 10 unidades
B1 — Chips	Snack (P03), tamaño “M” — 8 unidades
B2 — Choco	Snack (P04), tamaño “L” — 3 unidades

Además, el depósito de monedas debe iniciarse con un fondo de 10 monedas de cada denominación.

Ejemplos de ejecución

IMPORTANTE: Las capturas que se muestran a continuación pretenden ilustrar funcionalidades, pero en ningún caso se espera que su formato sea idéntico al del proyecto a desarrollar. Además, siempre que se haga de forma coherente, es posible alterar los flujos para adaptarlos al sistema desarrollado o facilitar la usabilidad.

Iniciar el sistema (menú principal)

```

=====
BIENVENIDO A EXPO-VENDING
=====

===== PRODUCTOS =====
POS  NOMBRE      PRECIO  STOCK
-----
A1   Cola        1.10EUR 5 ud
A2   Agua        1.00EUR 10 ud
A3   ---          ---     VACÍO
A4   ---          ---     VACÍO
B1   Chips       1.50EUR 8 ud
B2   Choco       1.80EUR 3 ud
B3   ---          ---     VACÍO
B4   ---          ---     VACÍO
C1   ---          ---     VACÍO
C2   ---          ---     VACÍO
C3   ---          ---     VACÍO
C4   ---          ---     VACÍO
D1   ---          ---     VACÍO
D2   ---          ---     VACÍO
D3   ---          ---     VACÍO
D4   ---          ---     VACÍO
=====

CREDITO ACTUAL: 0€
=====
1. Insertar Monedas
2. Seleccionar Producto (Comprar)
3. Devolver Crédito
8. Modo Administrador (Requiere PIN)

Seleccione una opción:

```

Realizar una compra

```

Seleccione una opción: 1

--- PANEL DE MONEDAS ---
A: 2.00€ | B: 1.00€ | C: 0.50€
D: 0.20€ | E: 0.10€ | F: 0.05€
S: Finalizar inserción
Pulse tecla de moneda: B
Crédito: 1.00€

--- PANEL DE MONEDAS ---
A: 2.00€ | B: 1.00€ | C: 0.50€
D: 0.20€ | E: 0.10€ | F: 0.05€
S: Finalizar inserción
Pulse tecla de moneda: B
Crédito: 2.00€

--- PANEL DE MONEDAS ---
A: 2.00€ | B: 1.00€ | C: 0.50€
D: 0.20€ | E: 0.10€ | F: 0.05€
S: Finalizar inserción
Pulse tecla de moneda: S
Crédito: 2.00€

=====
BIENVENIDO A EXPO-VENDING
=====

===== PRODUCTOS =====
POS  NOMBRE      PRECIO  STOCK
-----
A1   Cola         1.10EUR  5 ud
A2   Agua         1.00EUR  10 ud
A3   ---          ---      VACÍO
A4   ---          ---      VACÍO
B1   Chips        1.50EUR  8 ud
B2   Choco        1.80EUR  3 ud
B3   ---          ---      VACÍO
B4   ---          ---      VACÍO
C1   ---          ---      VACÍO
C2   ---          ---      VACÍO
C3   ---          ---      VACÍO
C4   ---          ---      VACÍO
D1   ---          ---      VACÍO
D2   ---          ---      VACÍO
D3   ---          ---      VACÍO
D4   ---          ---      VACÍO
=====

CREDITO ACTUAL: 2.00€
-----
1. Insertar Monedas
2. Seleccionar Producto (Comprar)
3. Devolver Crédito
8. Modo Administrador (Requiere PIN)

Seleccione una opción: 2
Código producto (Ej: A1): B1

>>> ¡Producto entregado: Chips! <<<
Cambio devuelto:
  1 x 0.50€
Total cambio: 0.50€

```


Modo administrador

```
1. Insertar Monedas
2. Seleccionar Producto (Comprar)
3. Devolver Crédito
8. Modo Administrador (Requiere PIN)

Seleccione una opción: 8

[SISTEMA DE SEGURIDAD]
Introduzca el PIN de administrador (3 intentos restantes): admin
PIN incorrecto. Inténtelo de nuevo.
Introduzca el PIN de administrador (2 intentos restantes): 1234
Acceso correcto. Cargando panel de control...

--- MODO ADMINISTRADOR ---
1. Reponer Stock (Sumar unidades)
2. Nuevo Producto (Configurar ranura)
3. Vaciar Ranura (Eliminar producto)
4. Ver Estado de la Caja (Monedas cambio)
5. Ver Recaudación (Ventas totales)
6. Ver Stock Detallado
7. Cambiar contraseña
0. Salir
Seleccione opción:
```

Modo administrador → Reponer Stock

```
--- MODO ADMINISTRADOR ---
1. Reponer Stock (Sumar unidades)
2. Nuevo Producto (Configurar ranura)
3. Vaciar Ranura (Eliminar producto)
4. Ver Estado de la Caja (Monedas cambio)
5. Ver Recaudación (Ventas totales)
6. Ver Stock Detallado
7. Cambiar contraseña
0. Salir
Seleccione opción: 1
Posición: A3
Posición no encontrada.

--- MODO ADMINISTRADOR ---
1. Reponer Stock (Sumar unidades)
2. Nuevo Producto (Configurar ranura)
3. Vaciar Ranura (Eliminar producto)
4. Ver Estado de la Caja (Monedas cambio)
5. Ver Recaudación (Ventas totales)
6. Ver Stock Detallado
7. Cambiar contraseña
0. Salir
Seleccione opción: 1
Posición: A1
Cantidad a añadir: 2
Stock actualizado. Nuevo total: 7
```

Modo administrador → Nuevo producto + Ver Stock Detallado

```

--- MODO ADMINISTRADOR ---
1. Reponer Stock (Sumar unidades)
2. Nuevo Producto (Configurar ranura)
3. Vaciar Ranura (Eliminar producto)
4. Ver Estado de la Caja (Monedas cambio)
5. Ver Recaudación (Ventas totales)
6. Ver Stock Detallado
7. Cambiar contraseña
8. Salir
Seleccione opción: 2
Posición (A1-D4): D3
ID: P05
Nombre: Pan
Tipo (1-Bebida, 2-Snack): 2
Tamaño (S/M/L): M
Cantidad inicial: 4
Producto añadido correctamente en D3.

--- MODO ADMINISTRADOR ---
1. Reponer Stock (Sumar unidades)
2. Nuevo Producto (Configurar ranura)
3. Vaciar Ranura (Eliminar producto)
4. Ver Estado de la Caja (Monedas cambio)
5. Ver Recaudación (Ventas totales)
6. Ver Stock Detallado
7. Cambiar contraseña
8. Salir
Seleccione opción: 6
===== PRODUCTOS =====
POS  NOMBRE      PRECIO  STOCK
-----
A1   Cola        1.10EUR  7 ud
A2   Agua         1.00EUR  10 ud
A3   ---          ---      VACÍO
A4   ---          ---      VACÍO
B1   Chips        1.50EUR  7 ud
B2   Choco        1.80EUR  3 ud
B3   ---          ---      VACÍO
B4   ---          ---      VACÍO
C1   ---          ---      VACÍO
C2   ---          ---      VACÍO
C3   ---          ---      VACÍO
C4   ---          ---      VACÍO
D1   ---          ---      VACÍO
D2   ---          ---      VACÍO
D3   Pan          1.50EUR  4 ud
D4   ---          ---      VACÍO
=====

```

Modo administrador → Eliminar producto + Ver Stock Detallado

```

--- MODO ADMINISTRADOR ---
1. Reponer Stock (Sumar unidades)
2. Nuevo Producto (Configurar ranura)
3. Vaciar Ranura (Eliminar producto)
4. Ver Estado de la Caja (Monedas cambio)
5. Ver Recaudación (Ventas totales)
6. Ver Stock Detallado
7. Cambiar contraseña
0. Salir
Seleccione opción: 3
Posición a eliminar: 81
Ranura B1 vaciada.

--- MODO ADMINISTRADOR ---
1. Reponer Stock (Sumar unidades)
2. Nuevo Producto (Configurar ranura)
3. Vaciar Ranura (Eliminar producto)
4. Ver Estado de la Caja (Monedas cambio)
5. Ver Recaudación (Ventas totales)
6. Ver Stock Detallado
7. Cambiar contraseña
0. Salir
Seleccione opción: 6

===== PRODUCTOS =====
POS  NOMBRE      PRECIO  STOCK
-----
A1   Cola        1.10EUR  7 ud
A2   Agua        1.00EUR  10 ud
A3   ---          ---      VACÍO
A4   ---          ---      VACÍO
B1   ---          ---      VACÍO
B2   Choco        1.00EUR  3 ud
B3   ---          ---      VACÍO
B4   ---          ---      VACÍO
C1   ---          ---      VACÍO
C2   ---          ---      VACÍO
C3   ---          ---      VACÍO
C4   ---          ---      VACÍO
D1   ---          ---      VACÍO
D2   ---          ---      VACÍO
D3   Pan          1.50EUR  4 ud
D4   ---          ---      VACÍO
=====

```

Modo administrador → Ver estado de la caja + Ver recaudación

```

--- MODO ADMINISTRADOR ---
1. Reponer Stock (Sumar unidades)
2. Nuevo Producto (Configurar ranura)
3. Vaciar Ranura (Eliminar producto)
4. Ver Estado de la Caja (Monedas cambio)
5. Ver Recaudación (Ventas totales)
6. Ver Stock Detallado
7. Cambiar contraseña
0. Salir
Seleccione opción: 4

--- ESTADO DEL DEPÓSITO (CAMBIO DISPONIBLE) ---
Valor: 2.00€ | Cantidad: 21 unidades
Valor: 1.00€ | Cantidad: 20 unidades
Valor: 0.50€ | Cantidad: 19 unidades
Valor: 0.20€ | Cantidad: 20 unidades
Valor: 0.10€ | Cantidad: 20 unidades
Valor: 0.05€ | Cantidad: 20 unidades
-----

--- MODO ADMINISTRADOR ---
1. Reponer Stock (Sumar unidades)
2. Nuevo Producto (Configurar ranura)
3. Vaciar Ranura (Eliminar producto)
4. Ver Estado de la Caja (Monedas cambio)
5. Ver Recaudación (Ventas totales)
6. Ver Stock Detallado
7. Cambiar contraseña
0. Salir
Seleccione opción: 5
*****
RECAUDACIÓN TOTAL: 1.50€
*****

```

5. CONSEJOS Y CONSIDERACIONES

- Es obligatorio que el proyecto se lleve a cabo siguiendo el **paradigma de programación Orientada a Objetos**. Además, se deben seguir todas las consideraciones y buenas prácticas relativas a la programación en general ya conocidas.
- Se **validarán los datos** que se consideren oportunos utilizando en cada caso el procedimiento que se crea más conveniente. Además, mediante el uso de **modificadores de acceso**, se debe evitar que se pueda acceder a todas las variables y métodos desde cualquier parte del código (utiliza los métodos necesarios para poder dar y recuperar el valor de las variables).
- El programa debe ser fácilmente **adaptable y modificable**. Además, se evitará, en la medida de lo posible, la presencia de código duplicado.
- El código estará correctamente **estructurado y ordenado** (se prestará atención a la indentación, los saltos de línea...). Los nombres de las variables y los métodos elaborados han de ser descriptivos, utilizando un estilo y formato homogéneo.
- El código incluirá **comentarios** suficientes para facilitar su seguimiento y comprensión; estos deberán ser breves y descriptivos, sin reflejar obviedades que se denoten de leer el código. (La calidad léxica y ortográfica de los comentarios debe ser adecuada al nivel académico en el que nos encontramos, sin faltas de ortografía y con una redacción clara y comprensible). Puedes utilizar comentarios que te ayuden durante el proceso de desarrollo del código y, cuando vayas finalizando las tareas, borrarlos. (No confíes en que te vas a acordar de todo de un día para otro, porque no, no te vas a acordar; y si, vas a perder mucho tiempo)
- Inicialmente puede parecer difícil, ya que el contenido de las clases y las etapas de trabajo no se encuentran especificados en detalle. Por ello es MUY RECOMENDABLE que **antes de comenzar** dediques tiempo a decidir cómo vas a abordar el proyecto, cómo lo vas a estructurar, cuál va a ser tu plan de trabajo... Esto puede parecerte una pérdida de tiempo, pero una buena planificación te será de gran ayuda para después poder avanzar de forma ágil y ordenada en la creación del código asociado al proyecto.
- **No intentes hacerlo todo de golpe**. Una estrategia muy habitual y utilizada en programación es dividir un problema grande en varios problemas pequeños y luego resolverlos uno a uno de forma independiente. Empieza programando y probando las etapas una a una, añadiendo la siguiente cuando te asegures que la anterior funciona según las especificaciones, así será más fácil.
- **No dejes el proyecto para el final**, ve realizándolo poco a poco a lo largo del tiempo. Va a haber momentos en que te atasques con un determinado problema; no desistas y te quedes parado, intenta realizar otra implementación y regresar más tarde y despejado a resolver el problema. Para acabar el proyecto la clave es avanzar, aunque no logres realizar con éxito una determinada implementación no abandones el proyecto e intenta realizar todas las demás.

- **IMPORTANTE:** Inicialmente vas a tener la sensación de que no sabes hacer nada (porque no tendrás los conocimientos suficientes para hacer el proyecto completo), no te preocupes, analiza el problema planteado y vete trabajando sobre aquellas partes que si sabes resolver (que las hay). Si poco a poco vas generando módulos funcionales, posteriormente “únicamente” tendrás que integrarlos en el programa. Es importante que aproveches el tiempo, ya que si esperas a tener todos los conocimientos para comenzar es muy posible que vayas muy justo de tiempo para la fecha de entrega definida.
- **Utiliza todos los conocimientos que tienes** para llevar a cabo el proyecto. Puedes investigar y utilizar soluciones ya creadas disponibles en internet, pero dedica tiempo a entenderlas y adaptarlas (en caso de que no seas capaz de entender un módulo de código, considera no lo utilizarlo, ya que si posteriormente debes modificarlo es muy posible que no seas capaz y eso será un problema). Además, en un proyecto de mayor extensión que los ejercicios de clase y que requiere relacionar muchos conceptos distintos, como este, copiar y pegar código sin conocer su funcionamiento puede acabar boicoteando tu propio desarrollo.
- Se valorará positivamente cualquier **mejora adicional** que se incluya siempre que esta sea coherente y se presente debidamente implementada, comentada y justificada. (Antes de ponerte con las mejoras cerciérate de que lo pedido se encuentra funcionando, de nada sirve tener un programa mejorado si este no cumple las especificaciones solicitadas).
- **Aprovecha las sesiones de control** fijadas para resolver tus dudas y obtener información acerca de si el proceso de desarrollo que estás siguiendo se ajusta a lo esperado. Es el momento de que las preguntas las hagas tú. (Puede ser interesante que te prepares todas aquellas cosas que quieres consultar y tomes tú el control de la situación. Intenta obtener toda la información que necesites, sin pasarte, no te vamos a hacer el proyecto; pero para no responder a algo ya estamos nosotras).
- **Prepárate la defensa**, es el momento de poner en valor tu trabajo y demostrar la dedicación que has empleado para sacar adelante el proyecto, hazlo ver. Intenta que tu defensa sea atractiva y diferente, sin que se vuelva un circo. (Aunque el proyecto no haya salido 100% como esperabas intenta ser constructivo y enfocar los problemas o aspectos a mejorar como oportunidades de seguir evolucionando).

Fechas a tener en cuenta:

- **Martes 24 marzo** → Sesión de control
- **Lunes 21 abril** → Entrega proyecto + informe (23:59)
- **Última semana de abril (día por concretar)** → Defensa proyecto

6. INFORME

Se deberá elaborar un informe indicando las fases de desarrollo del proyecto, tal y como se ha visto en a lo largo del curso. El informe debe empezar con una portada, índice y una hoja de control de versiones. En él aparecerán los siguientes apartados:

1. **Fase Análisis:** analizamos las necesidades y definimos los requisitos funcionales y no funcionales que debe cumplir.

En este apartado debemos añadir un diagrama de casos de uso, en el que se indican todas las acciones que puede realizar el usuario.

2. **Fase Diseño:** se traducen los requisitos en análisis de componentes (variables, métodos, datos necesarios, etc). El objetivo de esta fase es intentar describir todo lo que debo programar para cumplir los requisitos, pero sin escribir ni probar el código: cuantos métodos necesito, qué variables voy a utilizar, qué estructuras condicionales son más apropiadas, etc.

Ejemplo: Se implementará un método que recoja una variable numérica introducida por teclado y mostrará si dicha variable es par o impar.

3. **Programación:** ejecutamos lo planeado en la fase de diseño. No debemos copiar y pegar código, sino que debemos describir detalladamente qué es lo que hace cada método.

*Ejemplo: el método se llamará **parimpar** y será de tipo void. Utilizando un objeto tipo Scanner, se recogerá el valor introducido por teclado y se almacenará en una variable numérica. Se utilizará para validar el valor del resto obtenido al dividir la cantidad introducida entre 2 y, con la ayuda de una estructura condicional IF..ELSE, se comprobará que:*

- *En el caso de que sea 0 se entrará en el IF y se imprimirá por pantalla un mensaje indicando que el número es par.*
- *En cualquier otro caso, se entrará por el ELSE y se imprimirá por pantalla un mensaje indicando que el número es impar.*

En este apartado debemos añadir un diagrama de clases en el que se muestre la estructura de cada una de las clases de nuestro proyecto.

4. **Pruebas:** Se realizará un plan de pruebas, en el que se deben validar todos los requisitos funcionales que hemos indicado en la fase análisis (si es posible, podemos validar varios requisitos en la misma prueba). Se debe entregar un Excel en el que se indiquen todos los casos de prueba (podéis utilizar la plantilla disponible en el *Classroom*) y es obligatorio que se adjunten evidencias de que dicho plan de pruebas ha sido ejecutado.

Estas evidencias pueden hacerse mediante capturas de pantalla que se añadirán al informe (es posible que haya que realizar varias capturas para una misma prueba) o mediante un(os) vídeos que se almacenarán en una carpeta de Drive (el video puede contener texto de edición o voz en off si se considera oportuno), la cual debe ser accesible para las profesoras tanto de programación como del módulo optativo y el enlace para acceder a ella debe estar disponible en el informe. Solo será necesario evidenciar las pruebas que correspondan con los casos de prueba indicados en el diagrama del apartado Análisis.

Nota: Como máximo se pueden juntar **3 pruebas en el mismo** video siempre y cuando estén claramente diferenciadas.

7. EVALUACIÓN DEL PROYECTO

Requisitos mínimos

En ningún caso se podrá superar el proyecto (calificación ≥ 5.0) si no se cumplen los siguientes requisitos:

- Entrega de todos los materiales solicitados en tiempo y forma atendiendo a las especificaciones presentadas (revisar el apartado 8 de este mismo documento).
- Defensa del proyecto en la fecha fijada atendiendo a las especificaciones presentadas (revisar **Anexo: Defensa del proyecto** en este mismo documento). Tras la defensa y posteriores preguntas no debe quedar duda de la autoría del presente proyecto por parte del alumno.
- El código entregado compila y se ejecuta.
- El programa en ningún caso aborta su ejecución de forma abrupta y/o inesperada.
- El programa presentado se ajusta de forma general a las especificaciones dadas. (El programa hace lo que se espera de él).
- La estructura general del programa denota un dominio suficiente de los contenidos implicados en el mismo, independientemente del grado de consecución final.
- El proyecto presentado debe ajustar su estructura al paradigma de **programación orientada a objetos**, estructurándose en clases (debidamente ordenadas) y paquetes. No se corregirá ningún proyecto que no se ajuste al mencionado paradigma, independientemente de que compile y realice las funciones pedidas.

8. ENTREGA

La entrega de todos los materiales solicitados deberá realizarse ante de la fecha de entrega fijada haciendo uso de los medios indicados.

No se corregirán las entregas realizadas después de la fecha de entrega fijada, salvo causa de fuerza mayor debidamente justificada; ni aquellas que no respeten las condiciones de entrega establecidas. En estos casos la calificación asociada al proyecto/informe será de 0 puntos.

La no entrega del proyecto/informe supondrá una calificación de 0 puntos.

Programación

Se entregará, en la tarea habilitada para tal efecto en la clase asociada al módulo de la plataforma *Google Classroom* el proyecto completo exportado del IDE de desarrollo *Eclipse* como archivo comprimido (.zip) nombrado como **nombreAlumno_proy2Cuatri.zip**.

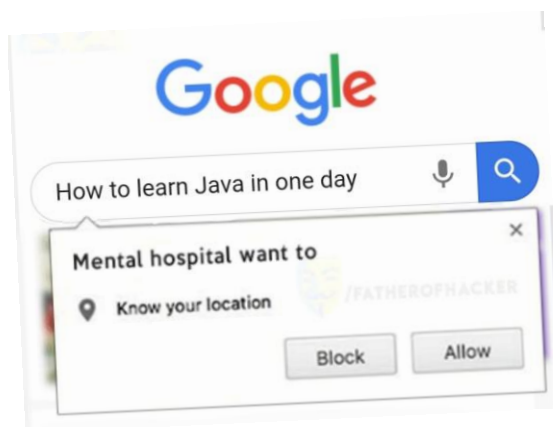
Entornos de desarrollo

Se realizará una entrega parcial en la que se deberán adjuntar todos los **diagramas (diagrama de clases y diagrama de casos de uso)** que deberán ir, posteriormente, incluidos en el informe final. Se entregarán una carpeta comprimida nombrada como **InformeProyecto_NombreApellidos** en una tarea habilitada en el Classroom de **Entornos de Desarrollo**.

Del mismo modo, el día de la sesión de control se validará que el diagrama de clases coincide con el desarrollo realizado en Eclipse.

Para la entrega final del **informe** se habilitará una tarea en la asignatura de **Entornos de desarrollo**. Se deberá entregar **en formato PDF** y nombrar como **InformeProyecto_NombreApellidos**. El resto de materiales (evidencias, plan de pruebas, etc.) deben ser accesibles desde el propio informe.

FECHA DE ENTREGA: MARTES 21 ABRIL (23:59)



ANEXO: DEFENSA DEL PROYECTO

Para finalizar la realización del proyecto se llevará a cabo una **defensa formal del mismo** en clase; en la que cada alumno defenderá su trabajo de forma individual.

La defensa consistirá en una breve exposición oral (**entre 6-8 minutos**) que podrá ser apoyada por diapositivas o cualquier otro material que se desee.

Al finalizar la exposición se deberá **tener disponible el código elaborado en disposición de ser ejecutado** si así se solicita. Además, el alumno quedará a disposición de profesores y compañeros para que le realicen distintas preguntas si así lo consideran.

En el ***apartado 6*** del presente documento, referente a la evaluación, se pueden consultar los criterios que se tendrán en cuenta para valorar la defensa.

El **orden de defensa** se publicará, como mínimo, 24 horas antes de la fecha establecida para la realización de estas.

FECHA DE DEFENSA: ULTIMA SEMANA DE ABRIL